

实验 1 关系数据库系统环境和数据库建立

班级: 07112304学号: 1120233329姓名: 陈墨霏

一、实验目的

安装和配置数据库管理系统, 设置系统环境变量, 选择合适的数据库管理工具, 为后续的数据库开发和操作提供良好的支持环境。

掌握关系数据库的基本操作, 熟悉数据库 DDL 语言的使用, 能够创建数据库、定义模式和基本表结构, 为后续数据库操作和应用开发打下坚实基础。

二、实验内容

1. 定义数据库, 采用中文字符集创建名为 TPCCH 的数据库
2. 定义模式, 在数据库 TPCCH 中创建名为 Sales 的模式
3. 定义基本表

表名	所含列	表含义
Region	地区编号, 地区名称, 备注	地区表
Nation	国家编号, 国家名称, 地区编号, 备注	国家表
Supplier	供应商编号, 供应商名称, 供应商地址, 国家编号, 供应商电话	供应商基本表
Part	零件编号, 零件名称, 制造商, 尺寸, 零售价格	零件基本表
PartSupp	零件编号, 供应商编号, 可用数量, 供应价格	零件供应联系表
Customer	顾客编号, 姓名, 国籍编号	顾客表
Orders	订单编号, 顾客编号, 订单日期, 订单总金额	订单表
Lineitem	订单编号, 零件编号, 供应商编号, 数量, 退货标记, 折扣[0.00, 1.00]	订单明细表

注: Part 表部分数据缺少尺寸。

三、实验步骤

3.1 实验环境及工具

本实验在 Windows 11 操作系统下进行，使用 MySQL 8.0.32 版本的数据库管理系统。

MySQL 是一个开源的关系数据库管理系统，广泛应用于 Web 应用程序和企业级应用中。它支持多种数据类型、索引、视图、存储过程等功能，具有高性能和可扩展性。可以从 [官方网站](#) 下载并安装。在安装时注意将 `mysql.exe` 添加到系统环境变量，以便在命令行中直接使用 MySQL 命令。

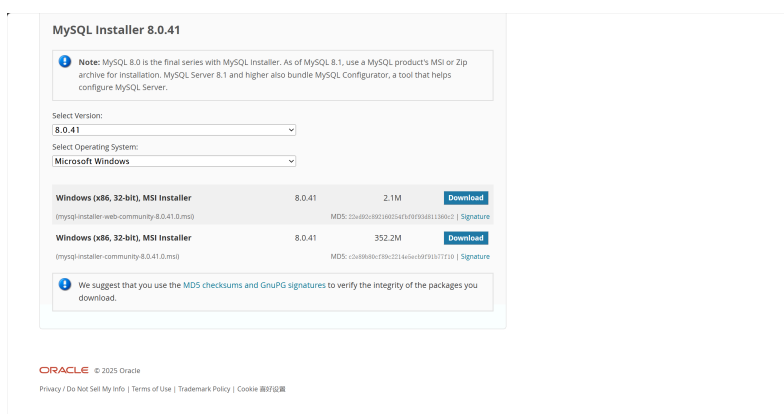


Figure 1: MySQL 数据库官方网站页面

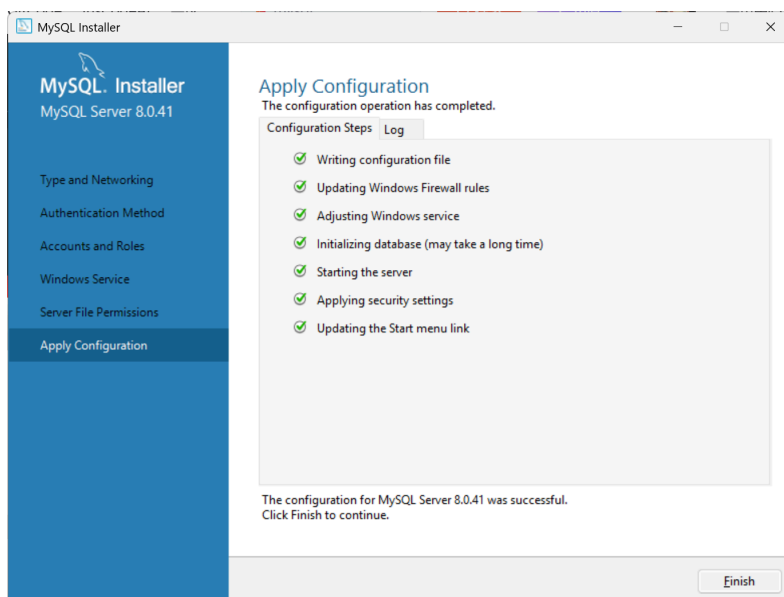


Figure 2: MySQL 安装界面

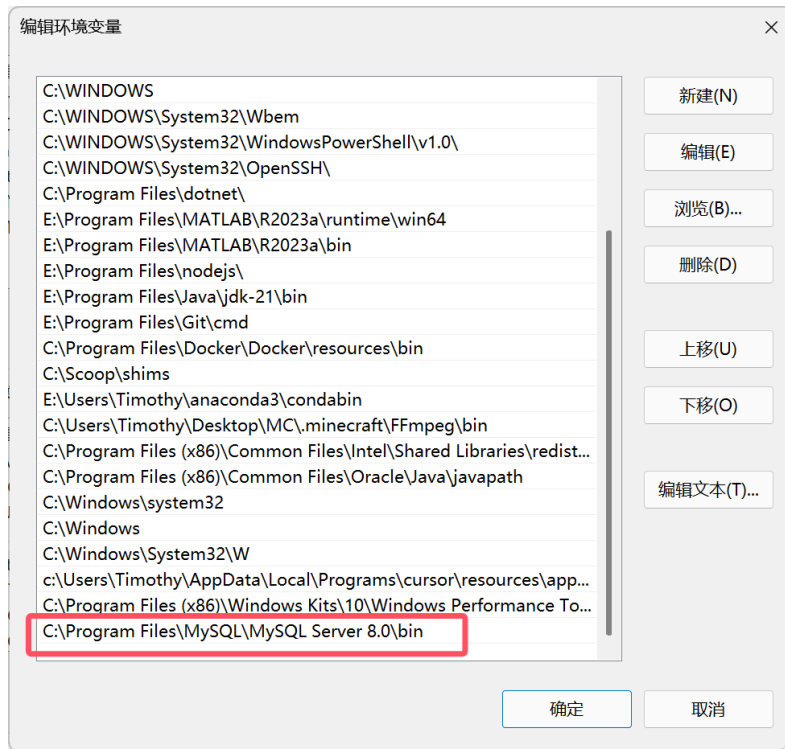


Figure 3: 系统环境变量配置

为了进行更加高效的数据库操作，我选择了 JetBrains 公司出品的数据库管理工具 **DataGrip** 进行操作。DataGrip 是一个跨平台的数据库管理工具，支持多种数据库系统，包括 MySQL、PostgreSQL、Oracle 等。它提供了强大的 SQL 编辑器、数据可视化和调试功能，能够帮助开发者更高效地进行数据库操作。

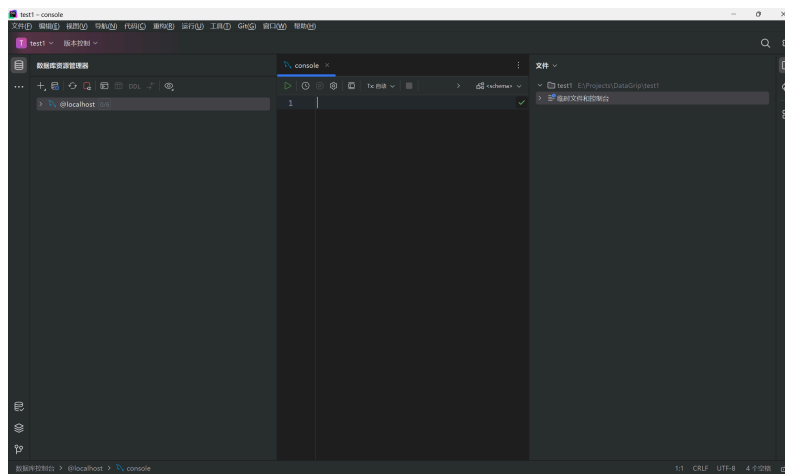


Figure 4: DataGrip 基础界面

3.2 连接到 MySQL 数据库

3.2.1 使用 MySQL Workbench 连接

MySQL Workbench 是 MySQL 官方提供的图形化管理工具，可以方便地连接和管理 MySQL 数据库。以下是使用 MySQL Workbench 连接到 MySQL 数据库的步骤：

打开 MySQL Workbench，可以看到下方 MySQL Connection 部分中已经创建好了一个本地实例（Local Instance）。直接点击该实例即可连接到本地 MySQL 数据库。

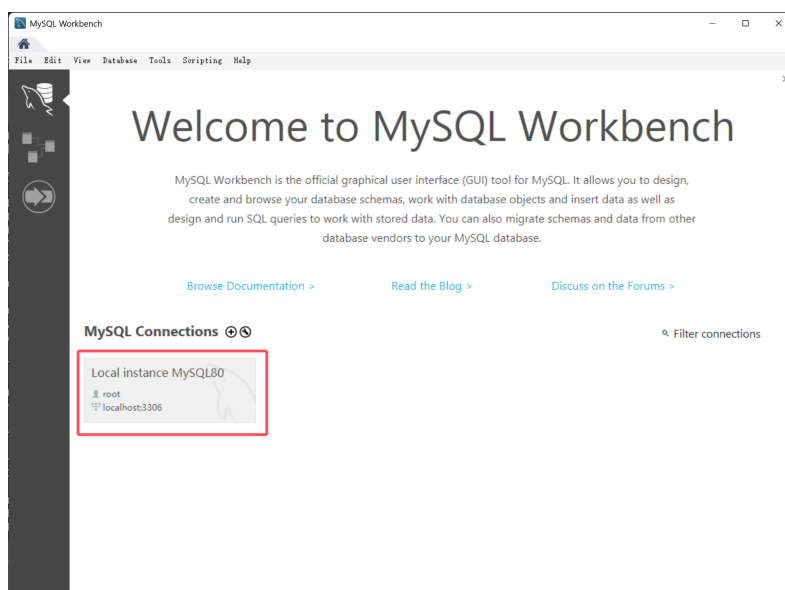


Figure 5: MySQL Workbench 连接到本地数据库

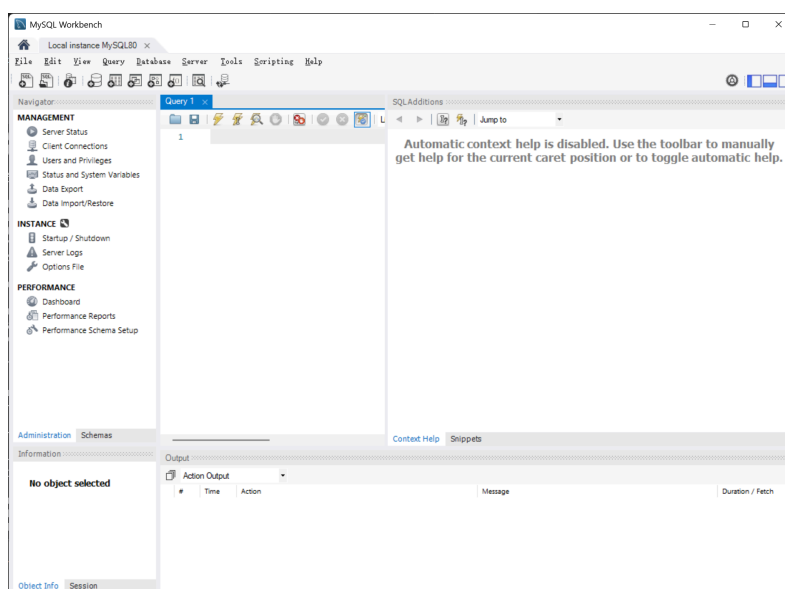


Figure 6: 连接成功

3.2.2 使用命令行连接

图形化界面可能更加简单易用，但是对于一些复杂操作的执行，命令行可能更加高效。以下是使用命令行连接到 MySQL 数据库的步骤：

打开命令行，输入以下命令连接到 MySQL 数据库：

```
mysql -h localhost -P 3306 -u root -p
```

其中，**-h**指定主机名，**-P**指定端口号，**-u**指定用户名，**-p**表示需要输入密码。连接成功后，可以看到 MySQL 的命令行提示符。或可直接输入以下命令连接到 MySQL 数据库：

```
mysql -u root -p
```

输入密码后即可连接到本地 MySQL 数据库。

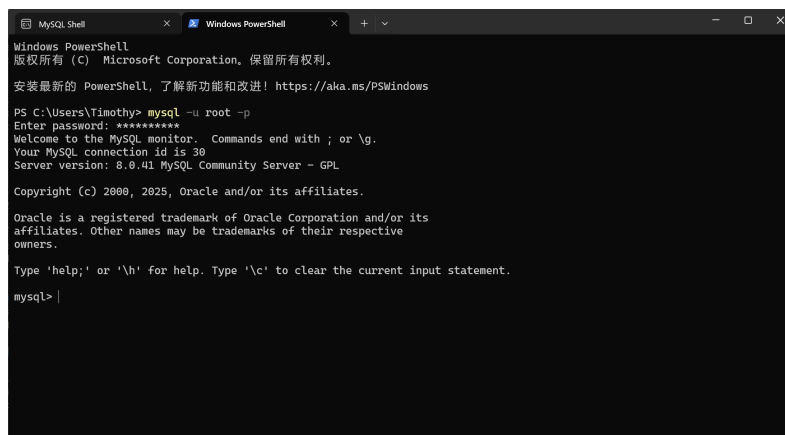


Figure 7: 命令行连接到 MySQL 数据库

3.2.3 使用 DataGrip 连接

在 DataGrip 中，点击左上角的**+**号，选择数据源，然后选择**MySQL**。在弹出的窗口中输入连接信息，包括主机名（此处为**localhost**）、端口号（此处为**3306**）、用户名和密码。点击**测试连接**按钮测试连接是否成功，如果成功则可以点击**确定**按钮保存连接信息。

3.3 数据库的建立

考虑到 DataGrip 集成了美观易用的图形化界面，以及简洁高效的命令行接口，接下来的实验将主要借助 DataGrip 进行 MySQL 数据库的建立和操作。

右键数据库，点击新建 — 查询控制台（或按快捷键 **Ctrl+Shift+Q**），创建一个新的查询控制台，在其中可以输入 SQL 语句进行数据库操作。

3.3.1 定义数据库/定义模式

MySQL 中没有模式（**schema**）的概念（**database**就是**schema**）。使用下列命令创建数据库并切换到该数据库：

```
-- create_db.sql

CREATE DATABASE TPCB CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE TPCB;
```

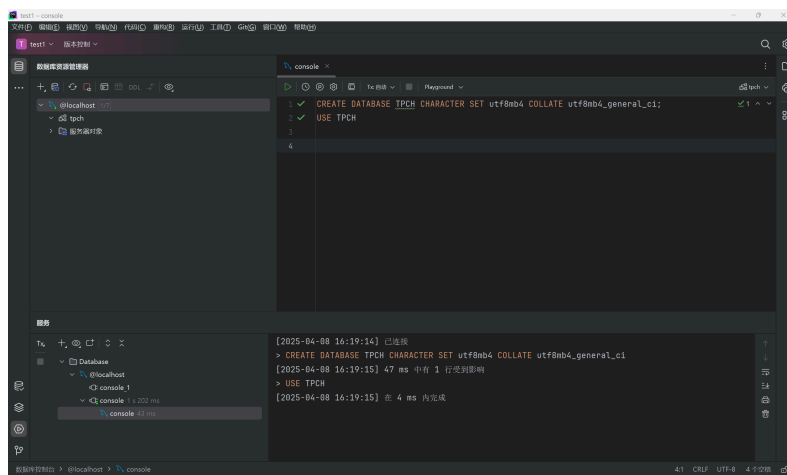


Figure 11: 创建数据库 TPCB

3.3.2 定义基本表

使用以下命令创建基本表：

```
-- create_table.sql

-- 地区表
CREATE TABLE Region (
    region_id INT PRIMARY KEY, -- 地区编号，作为主键
    region_name VARCHAR(50) NOT NULL, -- 地区名称，不能为空
    comment TEXT -- 备注
);

-- 国家表
CREATE TABLE Nation (
```

```
nation_id INT PRIMARY KEY, -- 国家编号, 作为主键
nation_name VARCHAR(50) NOT NULL, -- 国家名称, 不能为空
region_id INT, -- 地区编号
comment TEXT, -- 备注

-- 外键约束, 引用地区表的地区编号
FOREIGN KEY (region_id) REFERENCES Region(region_id)
);

-- 供应商基本表
CREATE TABLE Supplier (
    supplier_id INT PRIMARY KEY, -- 供应商编号, 作为主键
    supplier_name VARCHAR(50), -- 供应商名称
    address VARCHAR(100), -- 供应商地址
    nation_id INT, -- 国家编号
    phone VARCHAR(20), -- 供应商电话

    -- 外键约束, 引用国家表的国家编号
    FOREIGN KEY (nation_id) REFERENCES Nation(nation_id)
);

-- 零件基本表
CREATE TABLE Part (
    part_id INT PRIMARY KEY, -- 零件编号, 作为主键
    part_name VARCHAR(50), -- 零件名称
    mfgn VARCHAR(50), -- 制造商
    size INT, -- 零件尺寸, 可以为NULL
    retail_price DECIMAL(10, 2) -- 零售价格
);

-- 零件供应联系表
CREATE TABLE PartSupp (
    part_id INT, -- 零件编号
    supplier_id INT, -- 供应商编号
    avail_qty INT, -- 可用数量
    supply_price DECIMAL(10, 2), -- 供应价格

    -- 联合主键, 包含零件编号和供应商编号
    PRIMARY KEY (part_id, supplier_id),

    -- 外键约束, 引用零件基本表和供应商基本表
    FOREIGN KEY (part_id) REFERENCES Part(part_id),
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
);

-- 顾客表
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY, -- 顾客编号, 作为主键
    name VARCHAR(50), -- 顾客姓名
    nation_id INT, -- 国籍编号

    -- 外键约束, 引用国家表的国家编号
    FOREIGN KEY (nation_id) REFERENCES Nation(nation_id)
);
```



```
-- 订单表
CREATE TABLE Orders (
    order_id INT PRIMARY KEY, -- 订单编号, 作为主键
    customer_id INT, -- 顾客编号
    order_date DATE, -- 订单日期
    total_amount DECIMAL(10, 2), -- 订单总金额

    -- 外键约束, 引用顾客表的顾客编号
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

-- 订单明细表
CREATE TABLE Lineitem (
    order_id INT, -- 订单编号
    part_id INT, -- 零件编号
    supplier_id INT, -- 供应商编号
    quantity INT, -- 数量
    return_flag CHAR(1), -- 退货标记 (Y/N)
    discount DECIMAL(5, 2) CHECK (discount >= 0.00 AND discount <= 1.00), -- 折扣 (0.00到1.00之间)

    -- 联合主键, 包含订单编号、零件编号和供应商编号
    PRIMARY KEY (order_id, part_id, supplier_id),

    -- 外键约束, 引用订单表、零件基本表和供应商基本表
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (part_id) REFERENCES Part(part_id),
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
);
```

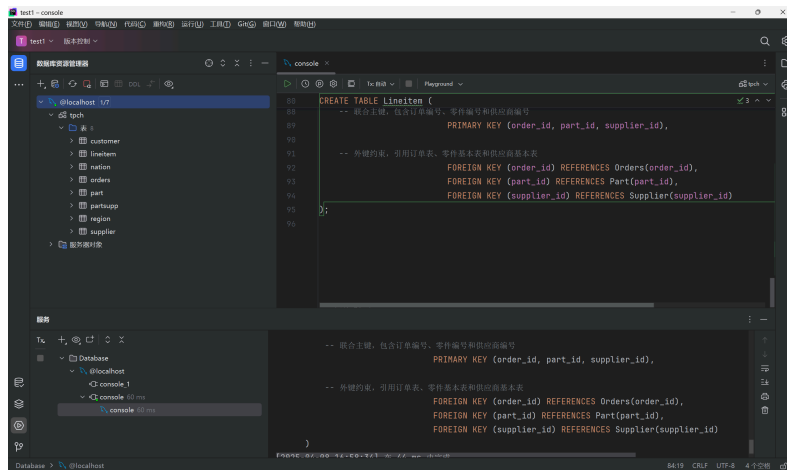


Figure 12: 创建基本表

四、实验结果及分析

本实验借助 MySQL 和 DataGrip 成功创建了 TPCB 数据库,并在其中定义了 Sales 模式和基本表。通过图形化界面和命令行两种方式,完成了数据库的创建和基本表的定义。

通过右键表—图—显示图（或按快捷键Ctrl+Alt+Shift+U）来显示当前情况下的表数据关系图。

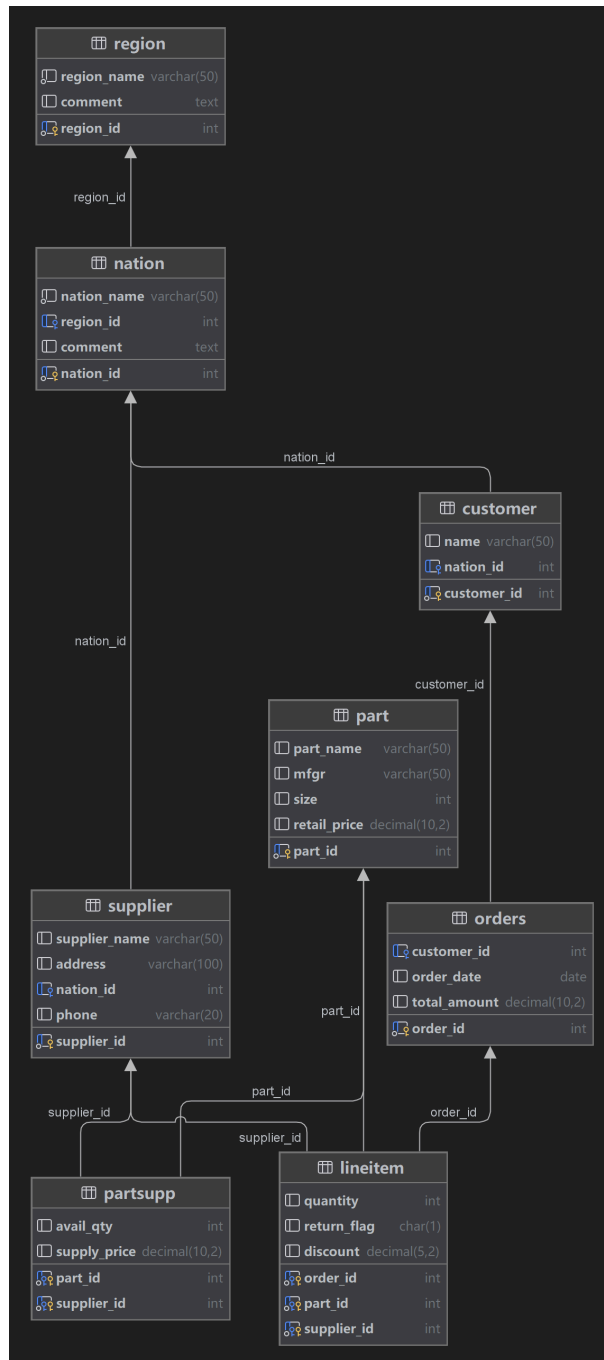


Figure 13: 表数据关系图

可以看到，创建的基本表中包含了各个表的主键和外键约束，确保了数据的完整性和一致性。这些约束不仅能够有效防止数据冗余和不一致，还能够通过外键关系建立表与表之间的关联，为后续的查询和操作提供了便利。此外，合理的字段设计和数据类型选择也为数据库的高效运行奠定了基础。

五、实验收获与体会

通过本次实验,我熟悉了 MySQL 数据库的安装与配置过程,掌握了基本的数据库操作方法,包括创建数据库、定义表结构及设置主外键约束等。借助 DataGrip 工具,我体验到了图形化界面带来的便捷性,同时也加深了对 SQL 语句的理解和应用能力。

此外,本实验让我认识到数据库设计的重要性。合理的表结构设计和字段约束不仅能提高数据存储的效率,还能确保数据的完整性和一致性。这为后续的数据库开发和复杂查询操作奠定了坚实的基础。

附录：程序清单及说明

以下是本实验中涉及的主要程序文件及其说明：

1. `create_db.sql`: 包含创建数据库 TPCH 的 SQL 语句，以及切换到该数据库的命令。
2. `create_table.sql`: 包含定义基本表的 SQL 语句，包括表结构、主键和外键约束等。

以上文件均已在实验中详细说明，并附有必要的注释以便理解和复现实验过程。